

Task 4 - Classification Models, Evaluation Metrics & Handling Imbalanced Data

AI / ML Internship | Telco Customer Churn Prediction

Objective

Task 4 moves from regression to classification. The goal was to build a binary classifier that predicts whether a customer will churn (leave the service) using the Telco Customer Churn dataset, and to evaluate it using metrics that actually matter for an imbalanced classification problem instead of relying on accuracy alone.

Dataset and Class Imbalance

The dataset has 7,043 customer records with 21 columns. After dropping the customerID column and converting TotalCharges to numeric, the data had no missing values left to handle. The target variable, Churn, was checked first for class balance.

Class	Count	Percentage
No Churn	5,174	73.5%
Churn	1,869	26.5%

Roughly 3 out of 4 customers did not churn. This means a model that always predicts "No Churn" would already score about 73% accuracy without learning anything useful, which is exactly why accuracy alone cannot be trusted here.

Why Accuracy Alone Is Misleading

The confusion matrix for the baseline Logistic Regression model on the test set (1,409 rows) was:

	Predicted No Churn	Predicted Churn
Actual No Churn	925	110
Actual Churn	162	212

Out of 374 customers who actually churned, the model correctly caught 212 of them (recall of 0.57) and missed 162. The overall accuracy was 0.807, which looks decent on the surface, but the recall for the churn class is the number that actually matters for the business. Missing 162 customers who were about to leave means missing 162 chances to offer a retention deal before they churned.

Metric Selection

For churn prediction, recall on the churn class is more important than overall accuracy or even precision, because a False Negative (missing an actual churner) is more costly to the business than a False Positive (flagging a loyal customer as a churn risk and offering them an unnecessary discount). F1-score is also useful as a single number that balances precision and recall, especially when comparing models on an imbalanced dataset.

Imbalance Handling

The `class_weight="balanced"` option in scikit-learn was used to make the model pay more attention to the minority churn class during training. The comparison below shows the effect on the test set:

Model	Precision (Churn)	Recall (Churn)	F1 (Churn)	Accuracy
Logistic Regression (baseline)	0.66	0.57	0.61	0.807
Logistic Regression (balanced)	0.51	0.79	0.62	0.740

Using `class_weight="balanced"` raised recall on the churn class from 0.57 to 0.79, meaning the model now catches far more of the customers who actually churn. This came at the cost of precision dropping from 0.66 to 0.51, since the model now also flags more customers who do not actually churn. Overall accuracy dropped from 0.807 to 0.740, but accuracy is not the metric that matters most here. The F1-score for the churn class stayed roughly the same (0.61 vs 0.62), showing this is a genuine precision-recall tradeoff rather than a clear improvement or regression.

Model Comparison: Logistic Regression vs Decision Tree

A Decision Tree Classifier was trained for comparison, without any depth restriction:

Model	Accuracy	Precision	Recall	F1-Score	AUC
Logistic Regression (baseline)	0.807	0.66	0.57	0.61	0.8418
Logistic Regression (balanced)	0.740	0.51	0.79	0.62	0.8414
Decision Tree (default)	0.726	0.48	0.48	0.48	0.6465

The Decision Tree had a train accuracy of 0.998 but a test accuracy of only 0.726, a clear sign of overfitting; it memorized the training data almost perfectly but generalized poorly. Its AUC of 0.6465 was also much lower than either Logistic Regression version, meaning it was worse at ranking customers by churn risk. Logistic Regression was more stable across train and test sets and is also easier to interpret, since the coefficients directly show how each feature affects churn probability.

Final Model Decision

Logistic Regression with `class_weight="balanced"` was selected as the more useful model for this churn problem, for the following reasons:

Higher recall on churn class: Catches 79% of customers who actually churn, versus 57% for the unweighted baseline. Missing fewer at-risk customers is more valuable than higher accuracy.

Comparable AUC: AUC of 0.8414 is almost identical to the baseline's 0.8418, showing the model's overall ability to separate churners from non-churners did not get worse, only the decision threshold behavior changed.

No overfitting: Unlike the Decision Tree, Logistic Regression does not show a large train-test performance gap.

Business fit: In churn prediction, a missed churner is a lost customer with no chance for retention, while a false alarm just means an unnecessary offer to a loyal customer. The cost of a false negative is higher, which is why recall was prioritized over raw accuracy or precision.

The unweighted Logistic Regression remains a reasonable choice if the business priority shifts toward minimizing false alarms instead, since it has noticeably higher precision. The right choice ultimately depends on which type of mistake is more costly for the business, which is a key takeaway from this task.

Dataset: Telco Customer Churn (7,043 rows, 21 columns). Models trained and evaluated using an 80/20 stratified train-test split.